



Campus Monterrey

Challenge Admin

Team 2

Flora Lindberg - A01831327

Suburam Ramatchandirin - A01831107

Rubens Emilien - A01831001

Cassandra Verdan - A01831154

Dr. Alfonso Avila

Design of Advanced Embedded Systems (Gpo 602)

Friday 7th of December, 2025

Introduction

Agriculture is one of the earliest human activities, but it is now at the center of some of the most pressing issues the world is confronted with: how to produce more food in a more efficient, sustainable, and lower-input manner. Industrial agriculture at large was transformed in recent years by GPS-tagged tractors, unmanned aerial vehicles, and smart sensors. However, small and medium-sized farmers, especially in countries like Mexico, generally cannot take advantage of these tools because they are expensive and complex.

In this project we will propose a low-cost and modular autonomous tractor guidance system with the purpose of making small-scale farmers and students enjoy the precision-farming advantages. By using sensors, embedded processors, and mobile applications, the system autonomously computes and drives optimized routes between field elements. In this way, we aim to conserve effort, fuel, and inputs lost, allowing operators to focus on value-added activities.

1. The name of the technological solution

We decided to call our solution **AgroCamino**. The name comes from two simple ideas: agro, for agriculture, and camino, for path. For us, it reflects the essence of the project which is helping farmers move with purpose and clarity in their fields.

Agriculture in the way we will view it here is all about making the best use of land, time, and resources. To do that, you need to follow a good camino, a path that is precise, efficient, predefined and reliable. Our system guides the tractor along this path, so that every movement is meaningful: no energy wasted, no land overlooked.

Choosing AgroCamino also carries a more human meaning. A camino is not just a line on a map, it's a journey; knowing where you are, where you need to go, and how to get there. That's exactly what we want to bring to small and medium farmers: the confidence that their work is guided, efficient, and moving in the right direction for more production and betterment of their journey in the fields regarding their products.

2. Product Description

AgroCamino was imagined with a simple vision: to make precision farming more accessible to small and medium farmers in Mexico. Instead of being a high-end technology reserved only for large industrial farms, the goal is to create an affordable tool that helps farmers work

more efficiently, reduce waste, and ultimately make their daily tasks less exhausting. The system is meant to bring the benefits of automation to people who normally could not afford it, while also serving as a learning platform for students.

The need for such a solution comes from the reality in the field. Today, many farmers still guide their tractors manually when planting, spraying, or fertilizing. This often leads to overlaps, wasted fuel, and uneven coverage, all while tiring out the operator. While big companies already use advanced GPS-guided systems, these solutions are far too expensive and complicated for smaller farms. AgroCamino tries to bridge this gap by offering a low-cost and modular prototype that shows how automation can be practical and realistic for everyday farming.

The main people who benefit from this project are small and medium farmers who want better results without investing in heavy, costly machinery. At the same time, the system is valuable in an academic context. For students and researchers, AgroCamino is a real testbed where they can experiment with embedded systems, sensors, and communication protocols while working on a problem that matters directly to local communities.

The product itself is built around a microcontroller (STM32) that acts as the brain of the system. It connects to a GNSS RTK module for precise positioning, an IMU to understand orientation, and LiDAR or ultrasonic sensors to detect obstacles. The tractor's movement is controlled with a motor encoder and CAN/J1939 communication, while an ESP32 handles Wi-Fi links with a mobile app. This allows a farmer to simply set field waypoints on the app, then let the tractor follow the route safely and reliably while monitoring its progress in real time.

Finally, from a business point of view, AgroCamino has several qualities that could make it marketable in the future. It uses affordable and widely available components, so the costs remain low. It is modular, so features can be added depending on the farmer's needs. And because it is easy to adapt, it could be used not only on farms but also in schools and training centers to teach smart agriculture.

Defined business goals:

Before thinking about markets and strategy, we asked ourselves a simple question: *what do we really want AgroCamino to achieve out in the real world?* The answer became our business goals. We don't want to just sell a product but also make farming easier, smarter, and more accessible while also creating space for learning and innovation. So here are a couple goals we came up with.

- **Keep it affordable**

Big companies already have fancy GPS tractors, but most farmers can't pay that kind of money. AgroCamino is built with cheap, reliable parts so small and medium farmers can finally get a piece of that precision tech without breaking the bank.

- **Make farm work easier and smarter**

With AgroCamino, tractors stay on track every time. That means no extra fuel burned, no double-spraying the same rows, and no long days of tiring manual steering. Farmers can work smarter, save money, and see better results in their fields.

- **Simple for people**

We don't want a system that only engineers can understand. With AgroCamino, the farmer just opens an app, drops some waypoints, and the tractor does the rest. It's tech that feels natural, not intimidating.

- **A “classroom” on wheels**

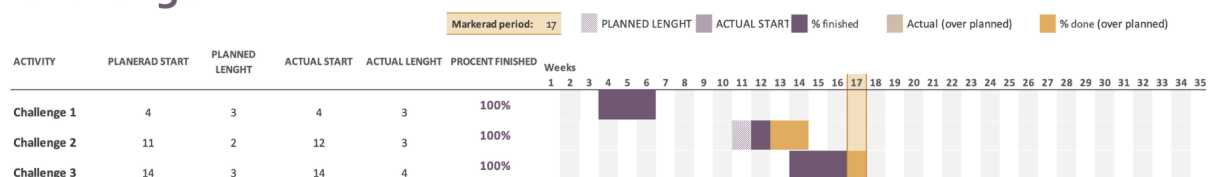
AgroCamino will serve as a type of “hands-on lab” because it will allow students and researchers to try out sensors, microcontrollers, and communication systems in a real setting, where their experiments connect directly to something that matters: growing food.

In this way, AgroCamino is more than just a prototype: it is a step toward making modern farming tools truly accessible.

3. Update the development and teamwork methodology

Gantt chart

Challenge



The Gantt chart shows the timeline for the project with the estimated time for each task, together with the actual time. The Gantt chart is divided into three different tasks, representing the three different parts of the challenge. For the first task, it is possible to see that we managed to finish the task on time. For the second task we did not follow the timeline completely since we experienced multiple problems with the hardware. This caused delay and contributed to not being able to finish the task on time. Since we got delayed on the second task, we were not able to start with the third task on time, meaning that the whole work got delayed. All of the assignments appeared throughout the course, which made it a bit difficult to plan ahead for the challenge. If we would have known all of the deadlines and assignments before, it would have been easier to do a better Gantt chart with more details and assignments for each of the persons instead of for the whole group. It could also have had more subtasks instead of having three large tasks.

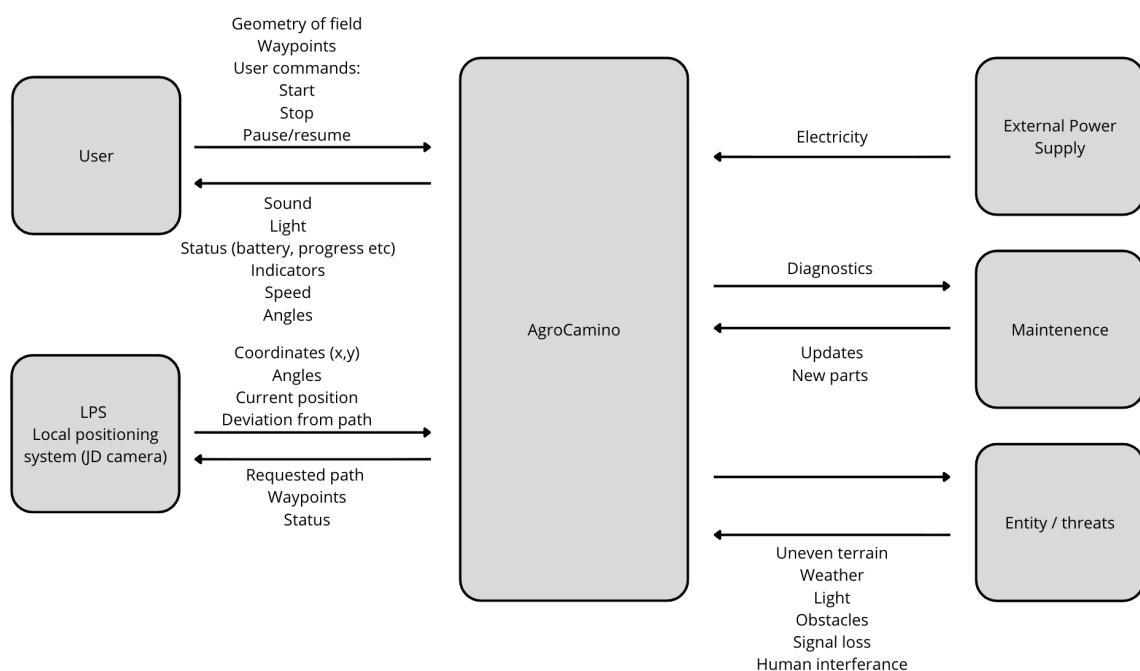
Reflection of the method when dividing the work

When dividing the work we tried to implement an agile method. Therefore, we divided the work so that some people were doing research and writing the reports and others were implementing the researched theory into the actual product. We did this to improve the efficiency of the process so that different group members could work on different things simultaneously. When dividing the work, we also based it mostly on our previous experiences and interests. If someone in the group had greater knowledge of something, it felt natural to let that person work on that specific task.

Proposed improvements for dividing the work

When reflecting back on how we divided the work, we would have done things differently if we were to do it again. The result of our way was that it was difficult to control the things you were not in charge of, and we had to blindly trust that the other people would finish the work on time. It also made it more difficult to give input on the work of that specific task when you were not the person in charge of it. On the other hand, not dividing the work at all would result in not being as efficient since everyone would have had to work on the same things all the time. If we would have had more specific roles in the group, we would have been more structured, resulting in a better result. We would have needed clearer deadlines within the group and a shared document to visually see all the things each person had to do. Overall, a better structure and planning is something that we will implement in our future projects.

4. Updated UML Context Diagram

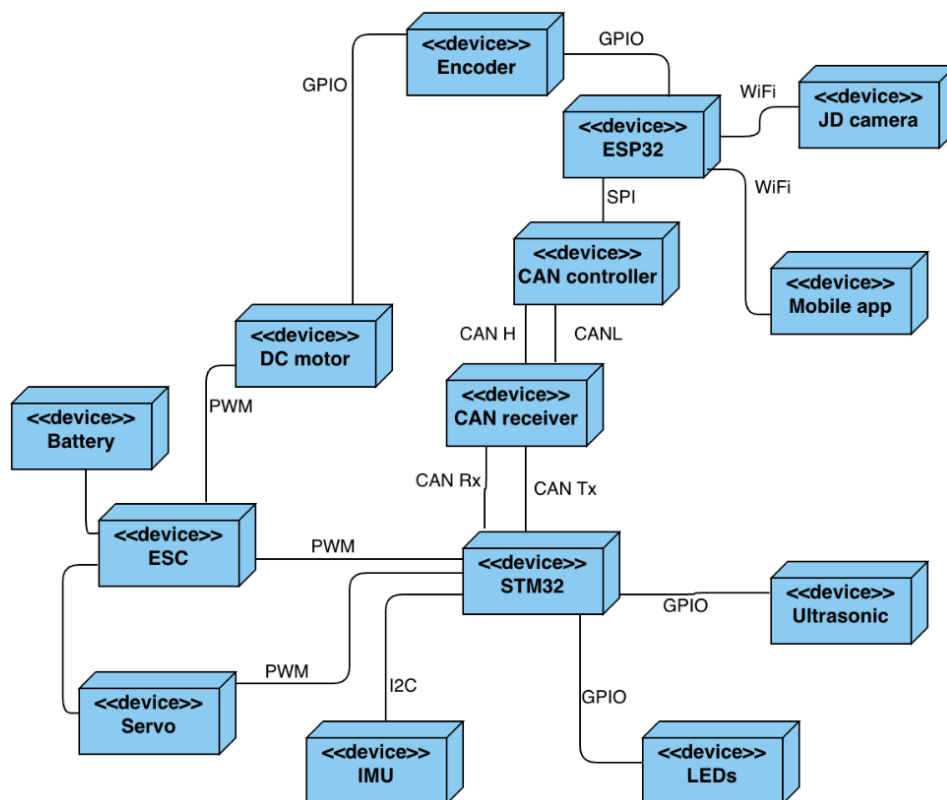


To be able to better understand how AgroCamino actually works in the real world, we sketched out its system context diagram. It shows the different elements that interact with the unit. From the farmer who uses a mobile application, through the sensors in the tractor, all the way down to the cloud where data can be analyzed.

At the center, AgroCamino brings together control, estimation, communications, diagnostics, and telemetry. Around it, we see the ecosystem external elements (JD camera, user, maintenance, battery, threats) that connect with the system.

So the context diagram is giving us a good sense of what talks to AgroCamino, what data goes in and out, and how all the components stay linked together to make accurate and efficient navigation possible.

5. Deployment diagram



While the context diagram shows who talks to AgroCamino, the deployment diagram shows how the system actually comes together and gets wired. At the heart of the design is the STM32 microcontroller, which acts as the brain. Around it, each component plays its role

- The battery and voltage regulator supply stable power to the entire system.
- The servo and ESC control the steering and motor, driven by PWM signals from the STM32.

- Sensors such as the IMU, ultrasonic modules, and encoder provide orientation, distance, and movement data to the controller.
- LEDs and the emergency stop add safety and system feedback.
- The ESP32 handles wireless communication, connecting the system to the mobile app for monitoring and control.
- All components communicate through protocols such as PWM, I²C, CAN, GPIO, and SPI to ensure reliable data flow.

So the deployment diagram gives us a pictorial sense of how the electronics, sensors, and actuators physically come together to implement AgroCamino in the real world.

6. Requirements Table

To turn AgroCamino from an idea into a working system, we need clear requirements. The following table outlines what the device is meant to do, the inputs and outputs it handles, and the functional and non-functional conditions it must meet. Together, these requirements set the foundation for both design and testing.

Purpose	The purpose of AgroRutaMX is to develop a smart device capable of generating and following optimized paths across agricultural fields in order to improve efficiency in agriculture logistics and field operations.
Inputs	• Waypoints such as coordinates where the tractor should go to. Optimized by software. • Current position of the device that is updated during the route of the tractor. Received from a GPS on the device. • Geometry of the field, by user input. • IMU, that can deliver acceleration and velocity angle. • Battery status

Outputs	· Path/movements of the tractor to be able to see if it follows the optimization from the device. Presents the speed, acceleration, angles etc by the motors. · LED indicators to show if the path is being followed. Green – correct, yellow – a bit off path, Red – out of range. · Tracking and analysis of the tractor's location over WiFi. · Error reports / diagnostics that can present error or status codes over WiFi or CAN. · Display feedback for the user on a screen or app.
Functional requirements (Functions)	FR1 – Path generator FR2- Path coverage, all waypoints are being passed. FR3 – Path following, making sure the paths are being followed. FR4 – Handling different shapes of the field. FR5 – Initial position handling FR6 – Navigation to origin after starting point is set. FR7 - Motor Control to control wheels and servo according to the path FR8 - Status Indication (LEDs) FR9 – Real time position updates FR10 - Diagnostics Reporting

Non functional requirements: performance	· The device shall generate paths within 5 seconds of receiving waypoints. · The device shall start following the path within 3 seconds after calculation. · The device shall read shaft position messages at a 10 ms repetition rate. · Signal processing of sensor data shall be completed within 10 ms to ensure timely control decisions. · The device should pass within 5cm of every viewpoint. · Position estimation from GPS, IMU, and shaft sensors shall have an accuracy of ± 5 cm. · The device shall not consume more than user inputted amount of liters of fuel per hour. · The device shall operate for at least 5 hours on a full fuel tank or battery charge · LEDs are updated within a rate of 10ms · The device shall wait for position coordinates at startup to ensure proper initialization. · The device shall move to the origin only after the position is determined.
---	--

Non functional requirements:

manufacturing cost

- **1 STM32 Development Boards (NUCLEO/Discovery)** — 1,200 MXN each → 1,200 MXN (Main controller, redundancy option)
- **1 ESP32 DevKit** — 300 MXN → 300 MXN (Wi-Fi telemetry)
- **1 GNSS RTK Module (u-blox NEO-M8P)** — 4,000 MXN → 4,000 MXN (Precision GPS)
- **1 IMU (MPU-9250 / BNO055)** — 800 MXN → 800 MXN (Orientation sensing)
- **6 Ultrasonic Sensors (HC-SR04 or similar)** — 50 MXN each → 300 MXN (Close-range detection)
- **1 Motor Encoder (Incremental, CAN capable)** — 1,200 MXN → 1,200 MXN (J1939 simulation)
- **1 DC Motor + ESC** — 2,500 MXN → 2,500 MXN (Drive system)
- **1 Servo Motor (Steering)** — 600 MXN → 600 MXN (Steering control)
- **1 Battery Pack (Li-ion 12V 20Ah) + BMS** — 2,500 MXN → 2,500 MXN (Power supply)
- **1 Voltage Regulator + Power Board** — 500 MXN → 500 MXN (5V/3.3V supply)

	· 2 nRF24L01+ RF Modules (optional) — 150 MXN each → 300 MXN (Backup comms) · 1 LED + E-Stop Button set — 200 MXN → 200 MXN (Diagnostics & safety) · 1 Chassis + Mounting Materials — 3,000 MXN → 3,000 MXN (Mechanical frame)
--	---

Non functional requirements: power	Battery, since it is a movable object that will not be able to be connected to a power outlet all the time.
Non functional requirements: physical size/weight	The device should not weigh above 10kg. The size should not exceed 40x40cm.

7. Testing process

Unit testing

	Testing CAN message			
Team	2			
Responsible	Alfonso Avila			
Description	The ESP32 sends a correct message to the STM32 over CAN, <u>UART</u> to print <u>message</u>			
Function	Check the CAN message over UART			
Components	ESP32, CAN transceiver, STM32 (with CAN Controller), PuTTY			
Hardware	ESP32 board, STM32 board, CAN transceiver, CAN bus wiring, PC (PuTTY)			
Procedure		Expected behavior with acceptance criteria	Actual behavior	Result passed or not
	Initial conditions Nothing is sent from the ESP32	Empty output, waiting for a message	PuTTY shows no output	OK
	Step 1: Connect <u>STM32</u> to the PC at <u>UART</u> and open PuTTY with <u>correct baudrate</u>	STM32 UART output is empty until a CAN message is received.	The STM32 displays nothing in Putty.	OK
	Step 2: Send test value 0x01 from the ESP32 into the CAN message.	STM32 shows CAN message and PuTTY prints value 0x01	0x01 shows in PuTTY	OK
Comments				

Integration testing

	Testing Shaft Position End-to-End			
Team	2			
Responsible	Alfonso Avila			
Description	Verify that turning the motor shaft updates the shaft-position value, which is then transmitted by ESP32 over CAN and received/processed by STM32, displayed via UART.			
Function	End-to-end shaft position measurement and reporting.			
Components	Encoder, ESP32, CAN controller, CAN transceiver, STM32, UART, PuTTY			
Hardware	Encoder, ESP32 board, CAN controller, CAN transceiver, STM32 board, CAN bus wiring, PC with PuTTY			
Procedure		Expected behavior with acceptance criteria	Actual behavior	Result passed or not
	Initial conditions Car is standing still	PuTTY shows waiting for incoming shaft-position data.	PuTTY is idle and ready.	OK
	Step 1: Rotate the shaft slowly by hand	PuTTY prints the value and indicates that the shaft has been turned.	Values appear in PuTTY, increasing or decreasing as the shaft turns.	OK
	Step 2: Rotate the shaft quickly	PuTTY prints values more frequently and the values are being changed faster.	Values appear more frequently in PuTTY, increasing/decreasing as the shaft is turned.	OK
Comments				

System testing

	Testing navigation system			
Team	2			
Responsible	Alfonso Avila			
Description	Verify that the car is following the path and passing the waypoints.			
Function	Path generation, waypoint following, motor control, sensor feedback and LED indication			
Components	STM32, ESP32, DC motor, ESC, Servo, Encoder, JD camera, IMU, CAN transceiver, CAN bus wiring, battery.			
Hardware	Fully assembled car, field with waypoints, laptop for monitoring, wireless communication			
Procedure		Expected behavior with acceptance criteria	Actual behavior	Result passed or not
	Initial conditions The car is placed on a random position in the field	The STM32 computes the path once the starting position is received.	-	-
	Step 1: Start the program and let the car follow the path	LEDs indicate that the waypoints are passed and the path is followed	-	-
Comments				

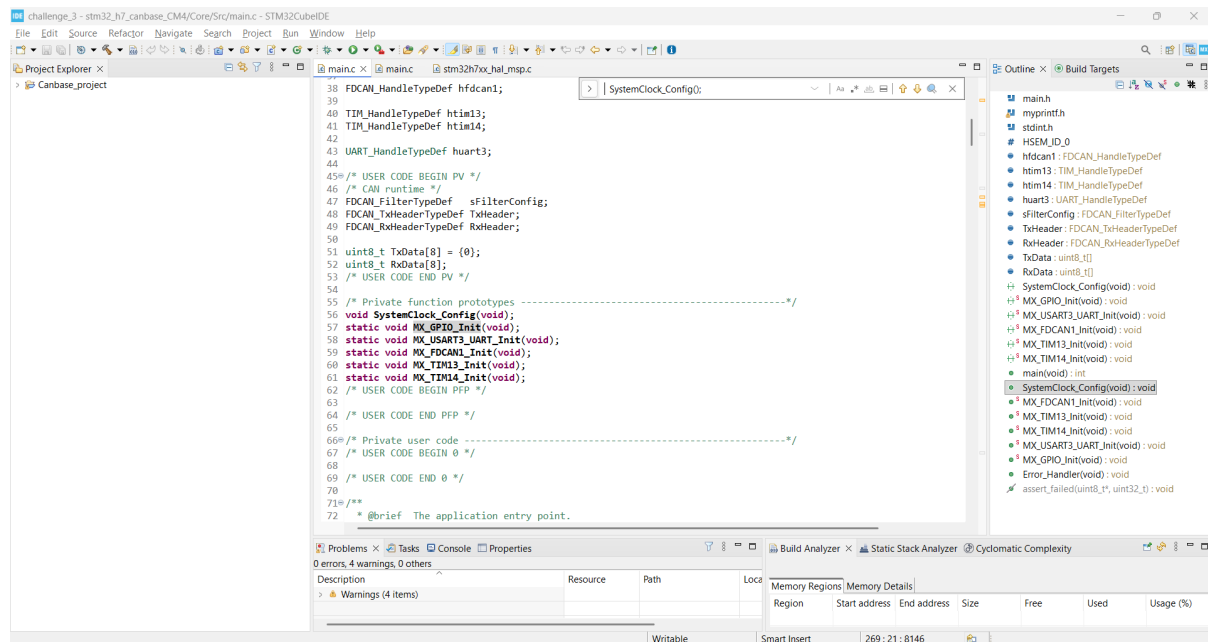
Since we were not able to implement the functionalities for the system test, there is no actual behaviour or results for that test.

8. Code Explanation

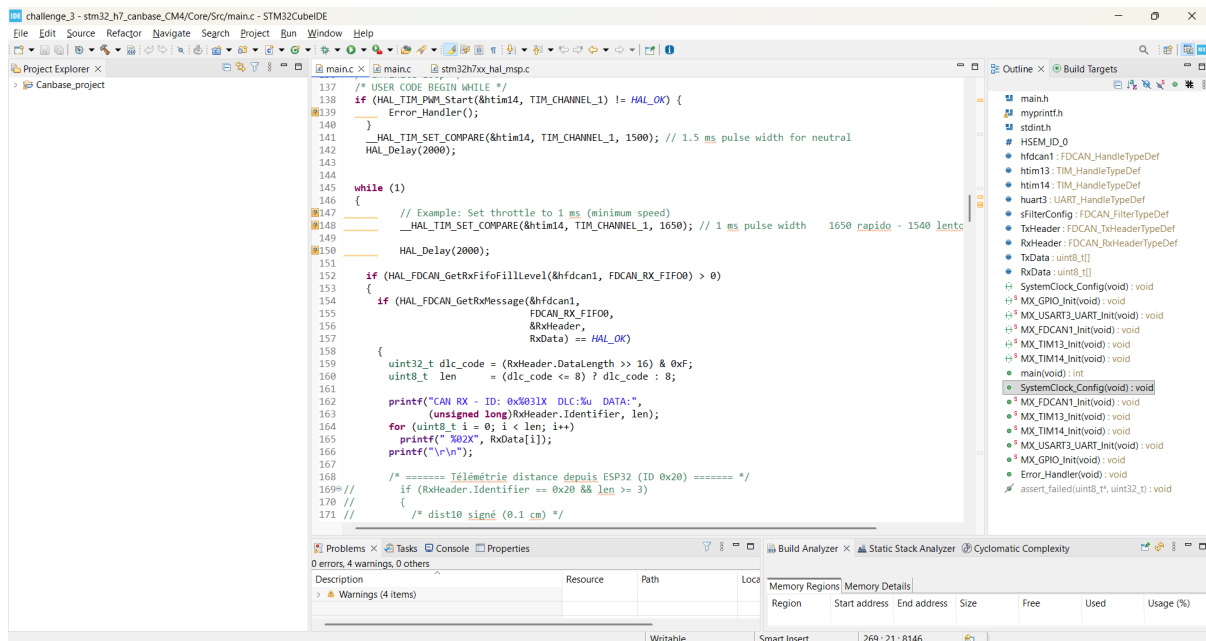
In this first part of the code, the goal is to set up everything the STM32 needs to run the different subsystems of our project: the CAN communication with the ESP32, the PWM signals for the

motor and the steering servo, the UART for sending information to the terminal, and the GPIOs for basic input/output functions.

To do this, the code begins by including several important files. One of the most useful is the HAL library, provided by ST. It makes development much easier: instead of working directly with the microcontroller's internal registers (which would be complex and hard to maintain), HAL offers ready-to-use functions to configure timers, CAN, UART, and GPIOs. This keeps the code much clearer and more manageable.



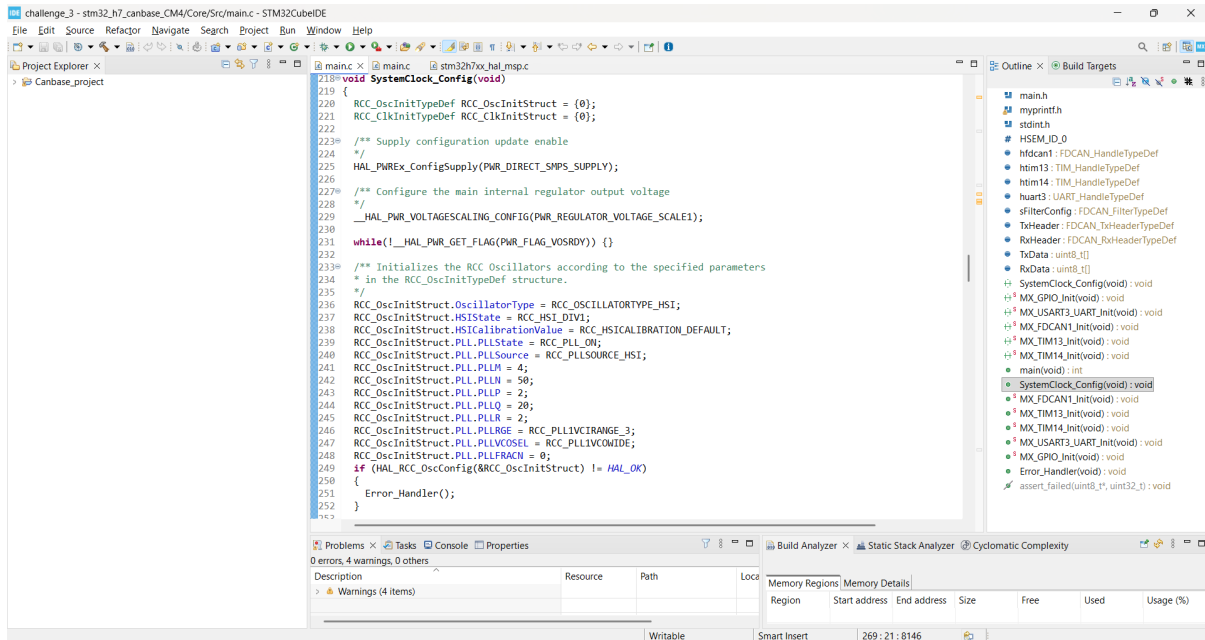
To begin, we use the HAL initialization is started with `HAL_Init()`. This function resets peripherals, configures the Flash interface, and sets up the SysTick timer used by HAL delays. The system clock is then configured through the function `SystemClock_config()`. This function sets up the internal oscillator and configures the PLL circuit that boosts the microcontroller's base clock to a higher speed so everything runs faster.. The PLL allows the microcontroller to achieve higher and more stable clock frequencies by multiplying the base oscillator signal.



If a message is present, it is read from the FIFO using `HAL_FDCAN_GetRxMessage()`. This part of the code is essential for communication between the two microcontrollers. The ESP32-C3 measures distance in real time using a rotary encoder and sends the distance value and a status flag through the CAN bus. The STM32 receives these values so it can eventually decide when to stop the motor or move to the next waypoint. When a CAN message is received, the STM32 first extracts the message ID, the number of bytes transmitted, and then each byte of the payload. All this information is printed over UART using `printf` so that the user can monitor communication through the serial terminal. The commented section in the code shows how the first three bytes can be interpreted: the first two combine into a signed 16-bit integer representing the distance in tenths of a centimeter, and the third byte is a flag indicating whether the target distance (one meter) has been reached. Although this handling block is commented out in the provided version, it clearly illustrates how the STM32 would normally process the distance data and react accordingly. In a full waypoint navigation implementation, reaching the target distance would automatically stop the motor, increment the waypoint counter, and trigger the transition to the next movement.

As a simple visual indicator, every successful CAN reception toggles a small LED on the STM32 board. This helps diagnose communication problems quickly, especially when debugging without a serial terminal.

Finally, a small delay is included at the end of the loop to prevent the processor from running at full speed unnecessarily. During this time, all hardware components continue working independently thanks to the HAL drivers: the PWM keeps running, the CAN controller continues listening, and the GPIO remains responsive.



After configuring the system clock and initializing all required peripherals, the code includes several essential hardware-setup functions that prepare each subsystem of the STM32 for its specific role in the tractor platform.

One of the most important components is the CAN bus, configured inside the function `MX_FDCAN1_Init()`. This function sets up the CAN controller so it can reliably communicate with the ESP32. First, the frame format is fixed to “classic CAN,” and the module is placed in normal operating mode. Then, the bit-timing parameters are calculated to obtain a 500 kbit/s bitrate, which is the same speed used by the MCP2515 CAN controller on the ESP32 side. The prescaler and timing segments are chosen based on the internal CAN clock frequency so that the communication remains stable even when the bus is shared by multiple nodes. The function also configures the CAN memory layout (message RAM), defines the number of receive and transmit slots, and creates a permissive filter that accepts all standard CAN identifiers. Finally, the function activates the CAN controller by calling `HAL_FDCAN_Start()`. Once this setup is complete, the STM32 is ready to receive any CAN frame coming from the ESP32.

The initialization functions for the timers, `MX_TIM13_Init()` and `MX_TIM14_Init()`, prepare the PWM outputs used to drive the electronic speed controller (ESC) and the steering servo. Both timers are configured with a time base of 1 μ s, allowing very precise control of the PWM pulse width. Their period is set to 20 ms (50 Hz), which is the standard frequency for RC actuators. The initial pulse width is set to 1500 μ s—a neutral position for both the motor throttle and the steering servo. By later adjusting the timer’s compare register, the code can increase or decrease the pulse duration, which directly translates into changes of motor speed or servo angle. These functions ensure that both actuators respond smoothly and consistently to control commands during navigation.

The function `MX_USART3_UART_Init()` configures the serial communication interface used for debugging and telemetry. The UART is set to 115200 baud in 8-N-1 mode, which allows the program to send formatted messages over the terminal using `printf()`. This interface is extremely useful for monitoring CAN traffic, validating system performance, and diagnosing issues during real-time testing.

Finally, the function `MX_GPIO_Init()` sets up all general-purpose I/O pins required by the system. It configures indicator LEDs as outputs, defines the push-button as an input, and assigns specific pins to alternate functions for peripherals such as timers or the CAN module. Proper GPIO configuration ensures that every physical signal—PWM outputs, CAN pins, and debugging LEDs—is electrically connected and behaves exactly as expected.

Overall, all these initialization functions work together to create a stable and functional environment for the tractor system. Once they are executed, the STM32 is fully prepared to read CAN messages, generate PWM signals for the actuators, transmit diagnostic information through UART, and interact with external sensors and processors.

In this project, the STM32 is in charge of several important subsystems that allow the vehicle to operate properly. Everything is configured through STM32CubeIDE, and the code relies on the HAL library to make hardware control easier without dealing directly with low-level registers. The first subsystem is PWM: TIM13 is used to generate the signal for the steering servo, where the pulse width between 1 ms and 2 ms sets the steering angle, and TIM14 generates the PWM signal for the ESC that drives the main motor. At startup, the ESC receives a neutral pulse of 1500 μ s, and then the speed of the motor is adjusted simply by changing the timer's compare value. Another key subsystem is the CAN interface (FDCAN1), which handles all communication between the STM32 and the ESP32 at 500 kbps. The STM32 receives telemetry such as the distance measured by the encoder and a flag indicating when a waypoint is reached, and each CAN message is read and processed inside the main loop using HAL functions that check the receive FIFO and extract the data. A UART interface is also initialized so the STM32 can print debugging information to a serial terminal, which is extremely useful during testing to confirm that CAN messages and PWM signals are working correctly. Finally, a few GPIO pins are used, especially for the onboard LEDs, so the code can toggle a light whenever a CAN message arrives, giving a simple visual confirmation that the communication is active. Altogether, the PWM, CAN, UART, and GPIO subsystems allow the STM32 to act as the main controller of the vehicle, managing both communication and physical actuation in a reliable way.

Conclusion

Throughout this project, we have stepped into a real-life challenge of system design. We have created a project from zero by stating the requirements, creating UML diagrams, assembling all components and implementing a functioning code matching the required functionalities. We have translated abstract ideas into concrete solutions, designed the system architecture, and implemented the logic needed to make all parts work together. The project gave us a hands-on experience in structuring a system from scratch and making sure that all pieces integrated smoothly.

By testing and refining our product, we strengthened our ability to develop complex systems and to translate theory into practice. The project also allowed us to understand how different parts of a system interact, how to troubleshoot when a problem arises, and how to work towards a final result. Finally, we learned how to approach real-world problems and improved our ability to work in teams.